



Lab Title: Insecure Deserialization

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS103 – Owasp Top 10 Vulnerabilities
Exploitation Techniques

Instructor Name: AMMAR AMINU

Date of Submission: 01/02/2026

Executive Summary

In this lab, I studied **Insecure Deserialization** and how it can be used to attack a web application. I created a vulnerable PHP page that unserialized user input without checking it. By submitting manipulated serialized data, I was able to change user roles and gain admin access. After that, I fixed the issue by replacing `unserialize()` with `json_decode()` and validating the input properly. This lab helped me understand how dangerous trusting user input can be and how to secure applications against this type of attack.

Lab Objective

The objective of this lab was to understand how insecure deserialization works, how attackers exploit it, and how to prevent it. I aimed to simulate a vulnerable PHP application, exploit the flaw by injecting malicious serialized data, and then apply secure coding practices to stop the attack.

Lab Environment

- Operating System: Kali Linux
- Web Server: Apache (XAMPP)
- Programming Language: PHP
- Browser: Google Chrome
- Tools Used: Text editor, local browser, PHP built-in functions

Task 1: Simulate Insecure Deserialization

Step 1: Setting up a Vulnerable PHP Application

- I created new folder [lab17_Insecure_Deserialization](#)
- Inside this folder, I created a file called `vulnerable.php`. I used the file to simulate insecure deserialization by unserializing data from an untrusted source.

```

(raph@kali)-[/opt/lampp/htdocs]
$ sudo mkdir lab17_Insecure_Deserialization
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs]
$ ls
applications.html      Lab2_HTMLInjection
bitnami.css            Lab4_CommandInjection
dashboard             lab8_BrokenAccessControl
favicon.ico           test.php
img                  vulnerable_app
index.php            webalizer
lab10_xss           week2_php_lab
lab17_Insecure_Deserialization

(raph@kali)-[/opt/lampp/htdocs]
$ cd lab17_Insecure_Deserialization

(raph@kali)-[/opt/lampp/htdocs/lab17_Insecure_Deserialization]
$ ls

(raph@kali)-[/opt/lampp/htdocs/lab17_Insecure_Deserialization]
$ sudo nano vulnerable.php

(raph@kali)-[/opt/lampp/htdocs/lab17_Insecure_Deserialization]
$ ls
vulnerable.php

```

Vulnerable.php

In this application, user input from a form is directly unserialized without any validation. Since the data comes from an untrusted source, an attacker can modify it to inject malicious objects or gain unauthorized access. This makes the application vulnerable to insecure deserialization attacks.

```
Session Actions Edit View Help
GNU nano 8.6 vulnerable.php *
<?php
session_start();

// Simulated class with a method that can be abused
class User {
    public $username;
    public $role;

    public function __construct($username, $role) {
        $this->username = $username;
        $this->role = $role;
    }

    // Simulated method that will be called during unserialization
    public function login() {
        echo "<h1>Welcome, " . $this->username . "!\</h1>";
        echo "<p>Your role is: " . $this->role . "</p>";
    }
}

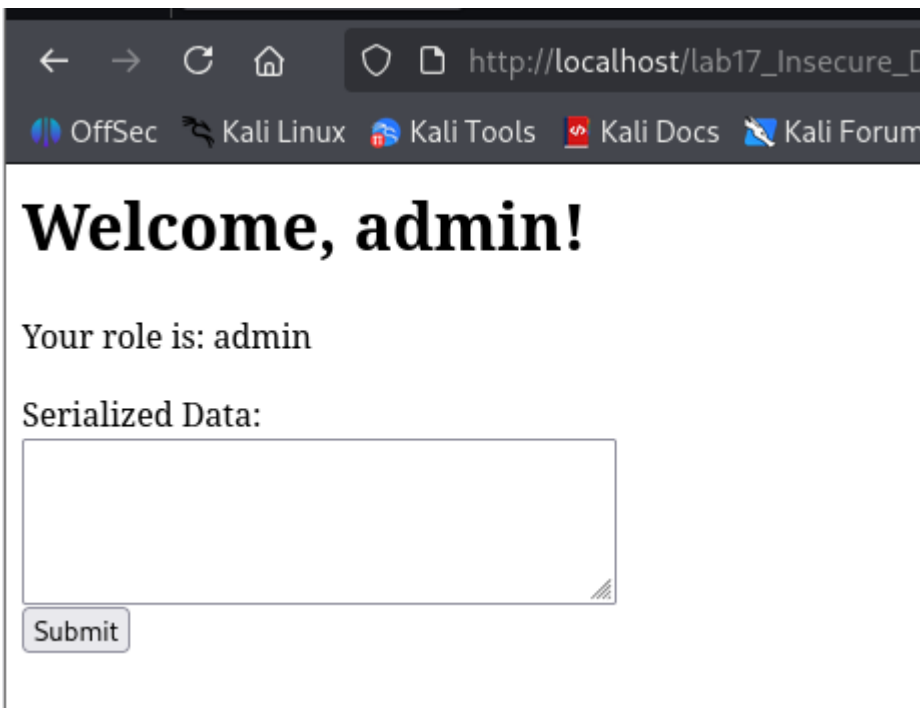
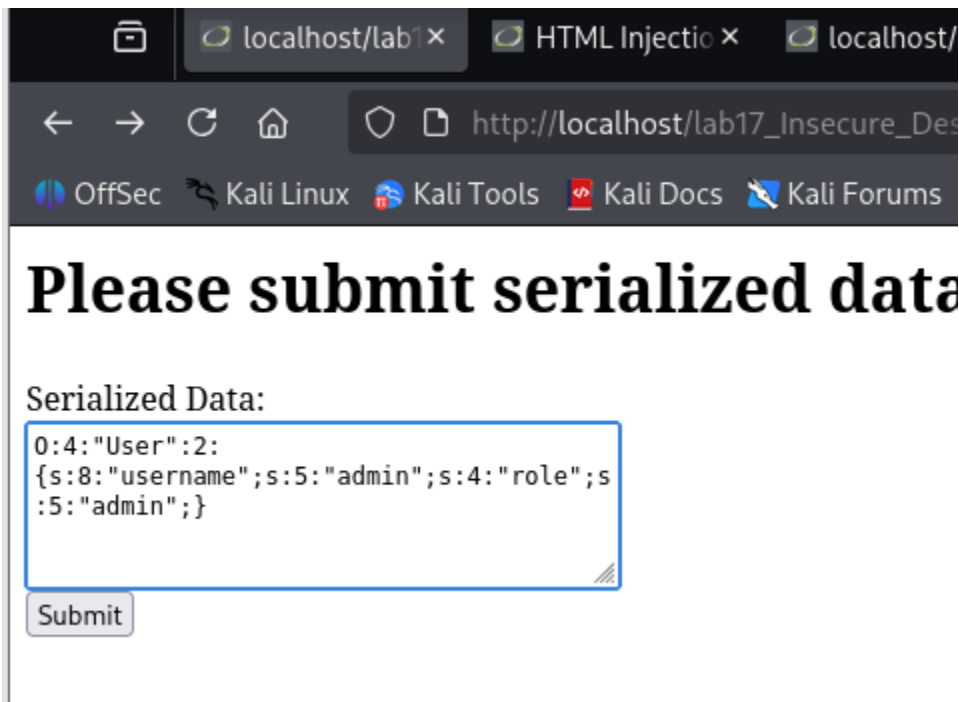
if (isset($_POST['serialized_data'])) {
    // Simulating unserialization from untrusted input
    $data = $_POST['serialized_data'];
    $user = unserialize($data);

    // Call the login method, which might be triggered on deseriali
    $user->login();
} else {
```

Step 2: Manipulate Serialized Data & Exploiting the Vulnerability

In the vulnerable.php application, I injected the malicious serialized payload

O:4:"User":2:{s:8:"username";s:5:"admin";s:4:"role";s:5:"admin";} from the lab material. The application accepted the payload as valid and created a user with an admin role without any verification.

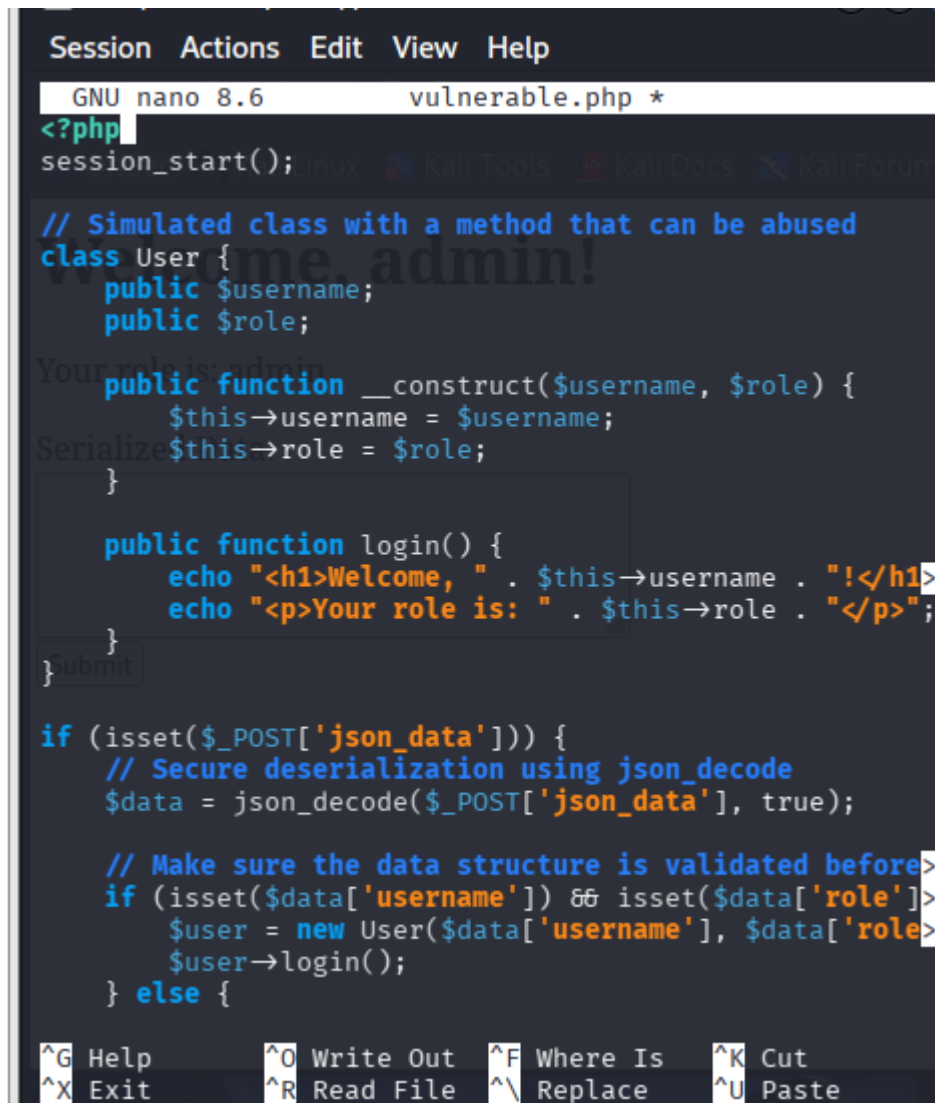


Task 3: Mitigating Insecure Deserialization

Step 1: Implemented Safe Deserialization

To prevent insecure deserialization, untrusted data should never be deserialized without proper validation. Instead of using PHP's `serialize()` and `unserialize()`, safer methods like `json_encode()` and `json_decode()` should be used. If serialization is required, the classes allowed for unserialization should be restricted using `allowed_classes`. In this lab, I refactored the application to fix the vulnerability by switching to `json_encode()` and `json_decode()`.

Updated vulnerable.php (safe Deserialization)



```
Session Actions Edit View Help
GNU nano 8.6 vulnerable.php *
<?php
session_start();

// Simulated class with a method that can be abused
class User {
    public $username;
    public $role;

    public function __construct($username, $role) {
        $this->username = $username;
        $this->role = $role;
    }

    public function login() {
        echo "<h1>Welcome, " . $this->username . " !</h1>";
        echo "<p>Your role is: " . $this->role . "</p>";
    }
}

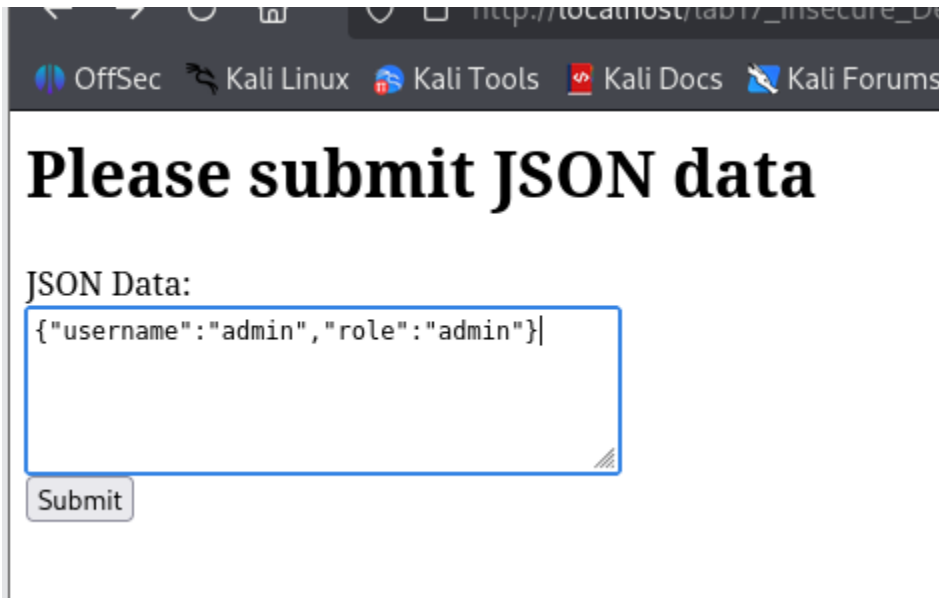
if (isset($_POST['json_data'])) {
    // Secure deserialization using json_decode
    $data = json_decode($_POST['json_data'], true);

    // Make sure the data structure is validated before
    if (isset($data['username']) && isset($data['role'])) {
        $user = new User($data['username'], $data['role']);
        $user->login();
    } else {

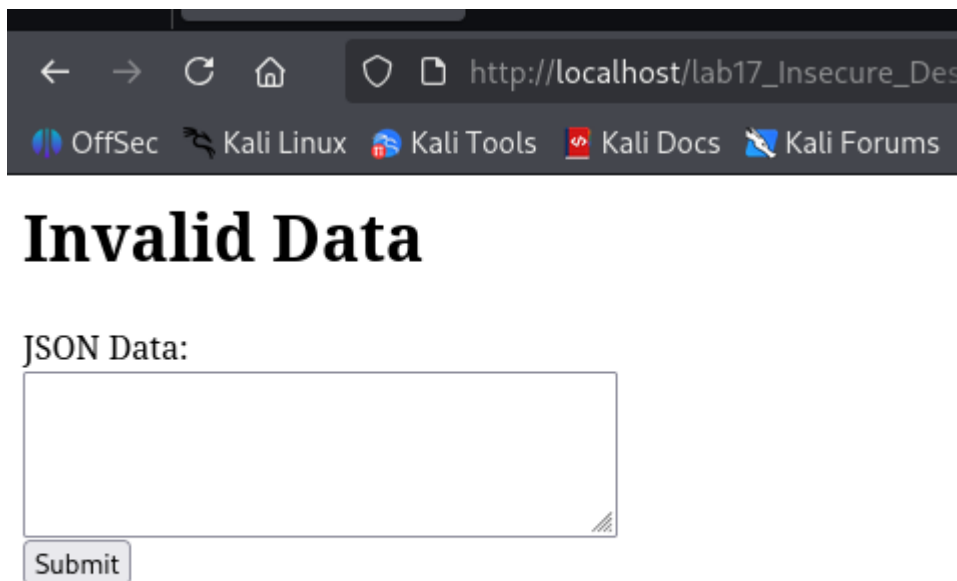
```

Step 2: Test the Mitigated Application

By submitting the same malicious serialized data before but now as a (JSON). The application treated it as invalid data.



A screenshot of a web browser window showing a web application interface. The browser's address bar displays the URL `http://localhost/lab17_insecure_De`. The browser's toolbar includes links to OffSec, Kali Linux, Kali Tools, Kali Docs, and Kali Forums. The main heading of the page is "Please submit JSON data". Below this heading, the text "JSON Data:" is followed by a text input field containing the JSON string `{"username": "admin", "role": "admin"}`. A "Submit" button is located below the input field.



A screenshot of a web browser window showing a web application interface. The browser's address bar displays the URL `http://localhost/lab17_insecure_Des`. The browser's toolbar includes links to OffSec, Kali Linux, Kali Tools, Kali Docs, and Kali Forums. The main heading of the page is "Invalid Data". Below this heading, the text "JSON Data:" is followed by an empty text input field. A "Submit" button is located below the input field.

Reflection Questions:

1. What is insecure deserialization, and how can it be exploited?

Insecure deserialization is when a program trusts data from a user and turns it back into an object without checking it. An attacker can change this data to trick the system into doing things it should not do, like giving admin access.

2. How did you exploit the vulnerability in the vulnerable application?

I exploited the application by sending modified serialized data through the form. The system trusted the data and logged me in as an admin without any checks.

3. How did the mitigations (switching to json_decode and validating input) prevent the exploitation?

Using json_decode() stopped the system from automatically creating objects. Checking the input also made sure only the correct data was accepted, so fake or malicious data could not work.

4. Why is it important to restrict the types of objects that can be unserialized?

It is important because attackers can create dangerous objects if there are no limits. Restricting objects helps prevent misuse and keeps the application safe.

Vulnerable.php

```
<?php
```

```
session_start();
```

```
// Simulated class with a method that can be abused
```

```
class User {
```

```
    public $username;
```

```
    public $role;
```

```
public function __construct($username, $role) {

    $this->username = $username;

    $this->role = $role;

}


// Simulated method that will be called during unserialization

public function login() {

    echo "<h1>Welcome, " . $this->username . "!</h1>";

    echo "<p>Your role is: " . $this->role . "</p>";

}

}


if (isset($_POST['serialized_data'])) {

    // Simulating unserialization from untrusted input

    $data = $_POST['serialized_data'];

    $user = unserialize($data);


    // Call the login method, which might be triggered on deserialization

    $user->login();

} else {

    echo "<h1>Please submit serialized data</h1>";

}

?>
```

```
<form method="POST">

    <label for="serialized_data">Serialized Data:</label><br>

    <textarea id="serialized_data" name="serialized_data" rows="5"
cols="40"></textarea><br>

    <button type="submit">Submit</button>

</form>
```

Vulnerable.php (Safe Deserialization)

```
<?php

session_start();

// Simulated class with a method that can be abused

class User {

    public $username;

    public $role;

    public function __construct($username, $role) {

        $this->username = $username;

        $this->role = $role;

    }

    public function login() {
```

```
        echo "<h1>Welcome, " . $this->username . "!</h1>";

        echo "<p>Your role is: " . $this->role . "</p>";

    }

}

if (isset($_POST['json_data'])) {

    // Secure deserialization using json_decode

    $data = json_decode($_POST['json_data'], true);


    // Make sure the data structure is validated before usage

    if (isset($data['username']) && isset($data['role'])) {

        $user = new User($data['username'], $data['role']);

        $user->login();

    } else {

        echo "<h1>Invalid Data</h1>";

    }

} else {

    echo "<h1>Please submit JSON data</h1>";

}

?>

<form method="POST">

    <label for="json_data">JSON Data:</label><br>
```

```
<textarea id="json_data" name="json_data" rows="5" cols="40"></textarea><br>
<button type="submit">Submit</button>
</form>
```

Key Takeaways

- Trusting user input can lead to serious security issues.
- Insecure deserialization allows attackers to change data and gain unauthorized access.
- Using `unserialize()` on untrusted data is dangerous.
- Switching to `json_decode()` and validating input helps prevent attacks.
- Secure coding practices are important for protecting web applications.

Conclusion

This lab helped me understand how insecure deserialization works and why it is risky. I learned how attackers can manipulate serialized data to gain access or change system behavior. By fixing the application using safer methods and proper validation, I was able to prevent the attack. This knowledge will help me write more secure code and identify similar vulnerabilities in real applications.